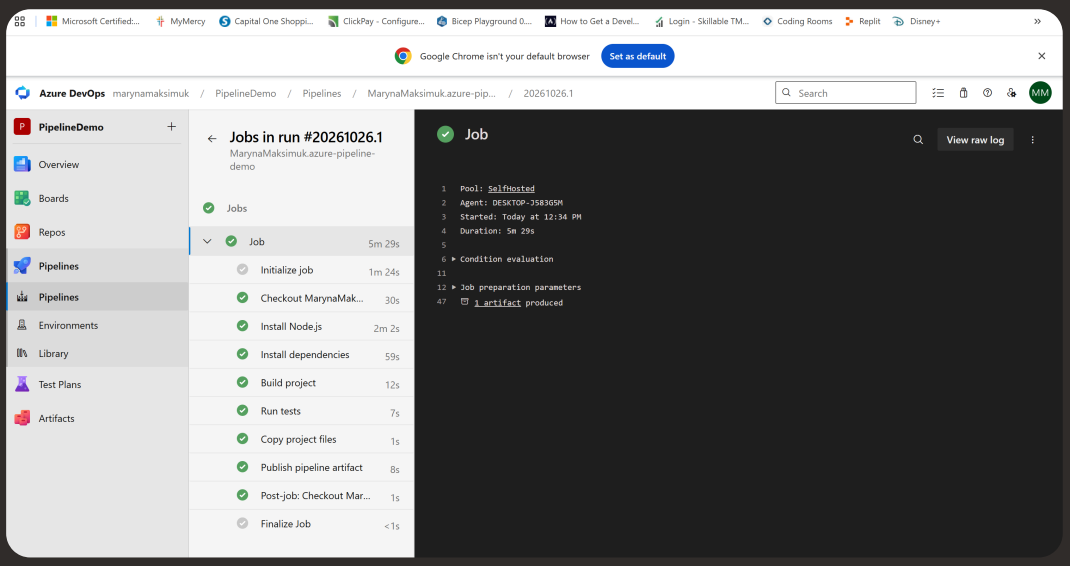
BUILD A CI/CD PIPELINE WITH AZURE DEVOPS

By Maryna Maksimuk | October 2026



Project Overview: Automating CI/CD with Azure DevOps

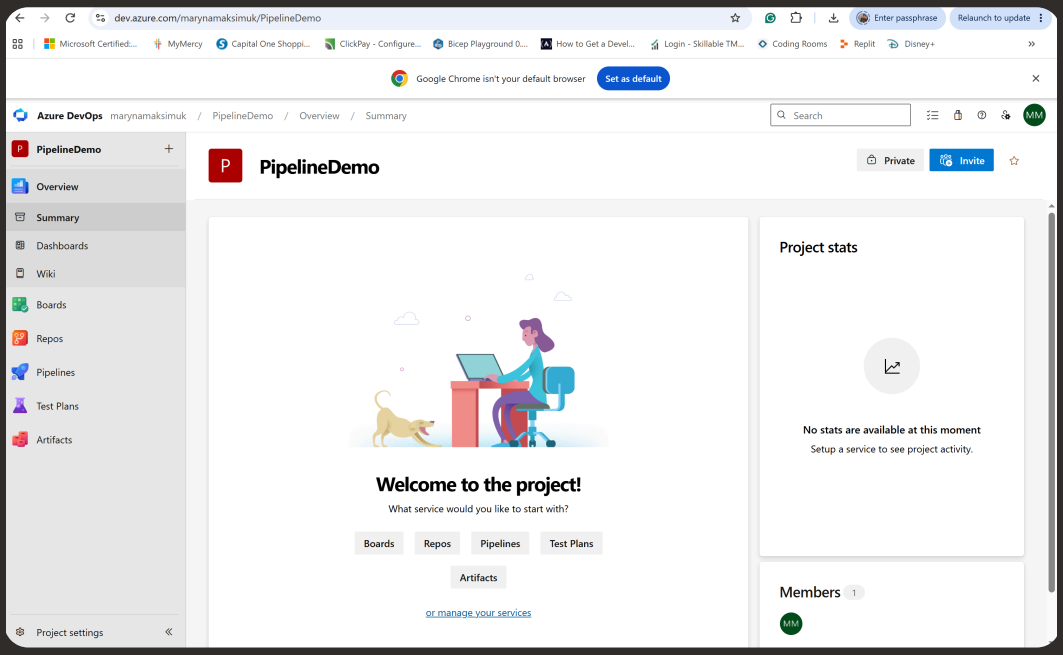
Goals and motivation

In this project, I'll set up an Azure DevOps CI/CD pipeline for a Node.js application. I'll configure a self-hosted build agent on my Windows machine, create a YAML pipeline to automatically build, test, and package the application whenever code is pushed to the main branch, and add separate Build and Deploy stages with a manual approval gate before deployment.

Setting Up the Development Environment

Planning the setup

In this step, I'll prepare my Windows environment by installing Git and Node.js, create a GitHub repository for my application, and clone it locally. I'll also set up an Azure DevOps organization and project to manage the repository's CI/CD pipeline.



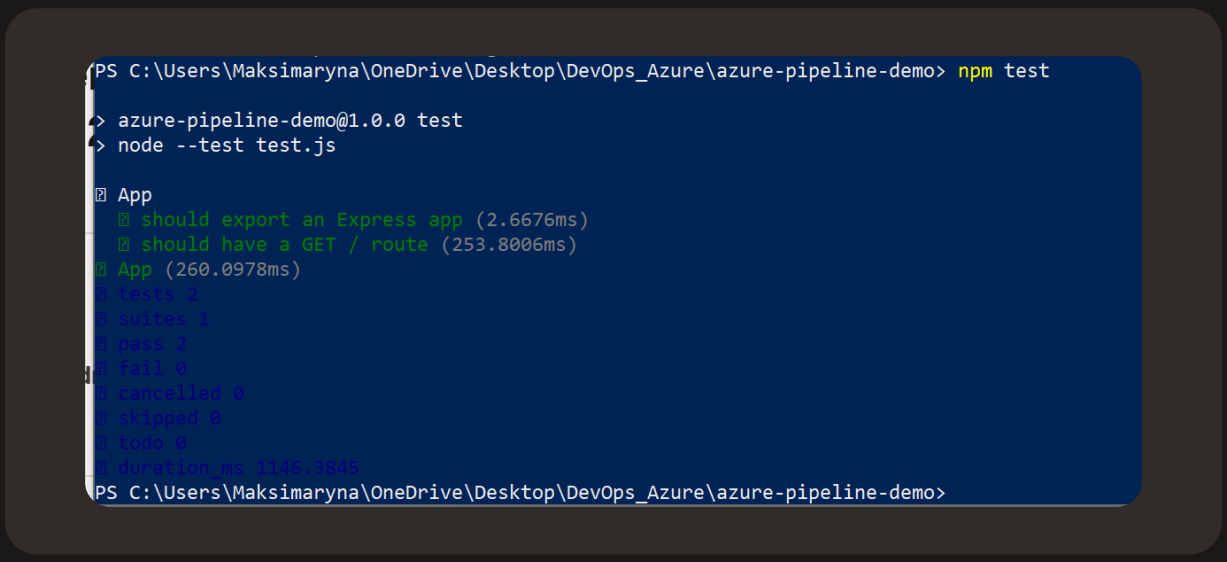
Tools installed and accounts created

In this step, I installed Git and Node.js on my Windows machine. I also created an Azure DevOps organization and project to manage my application and set up the CI/CD pipeline.

Building the Node.js Application with Automated Tests

Approach and objectives

In this step, I'll create a Node.js project and build a simple web server using Express. I'll write automated tests using Node.js's built-in test runner, run the tests locally to make sure everything works correctly, and then commit and push my code to GitHub.



Application behavior

My server responds with "Hello from Azure Pipelines!" when someone visits the root URL (/).

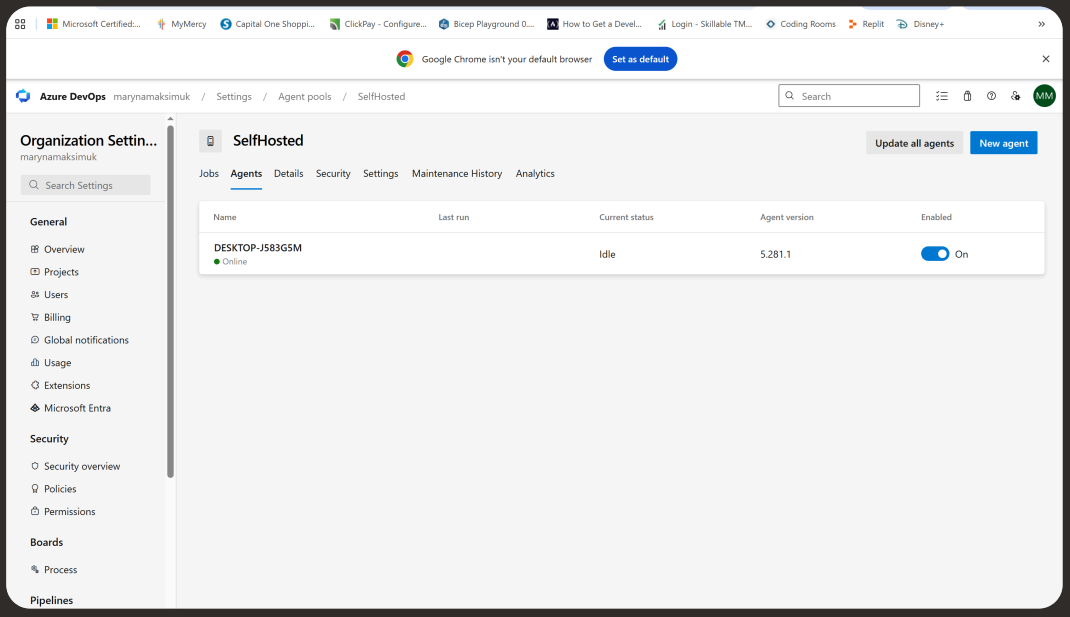
Configuring a Self-Hosted Build Agent

Why a self-hosted agent

In this step, I'll set up a self-hosted build agent on my Windows machine so Azure DevOps can run pipeline jobs locally. I'll create an agent pool, generate a Personal Access Token (PAT) for authentication, then download, configure, and start the agent so it's ready to execute my CI/CD pipeline.

Personal Access Token scope

I gave my Personal Access Token the Agent Pools (Read & manage) scope so I could authenticate, configure, and register my self-hosted agent with Azure DevOps and manage its connection to the agent pool. I used this scope because the agent needs permission to connect to Azure DevOps and execute pipeline jobs.



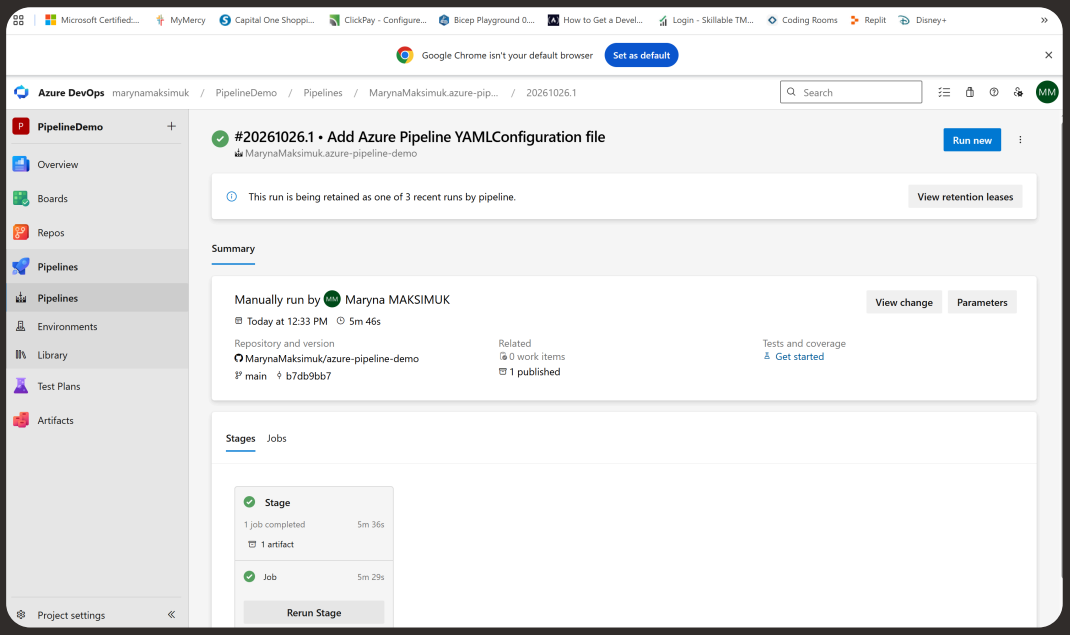
Agent lifecycle considerations

If I close the PowerShell window running run.cmd, the self-hosted agent will stop running. Azure DevOps will show the agent as offline, and it won't be able to execute pipeline jobs until I start it again.

Writing and Running the YAML Pipeline

Pipeline design

In this step, I'll create an azure-pipelines.yml file to automate building and testing my Node.js application. I'll add a publishing task to package the build output as a pipeline artifact, then create and run the pipeline in Azure DevOps to verify that the build, tests, and artifact publishing all work successfully.



Understanding the trigger configuration

The trigger: - main section tells Azure Pipelines to automatically run the pipeline whenever new code is pushed to the main branch in GitHub. This ensures my Node.js application is automatically built and tested after each commit.

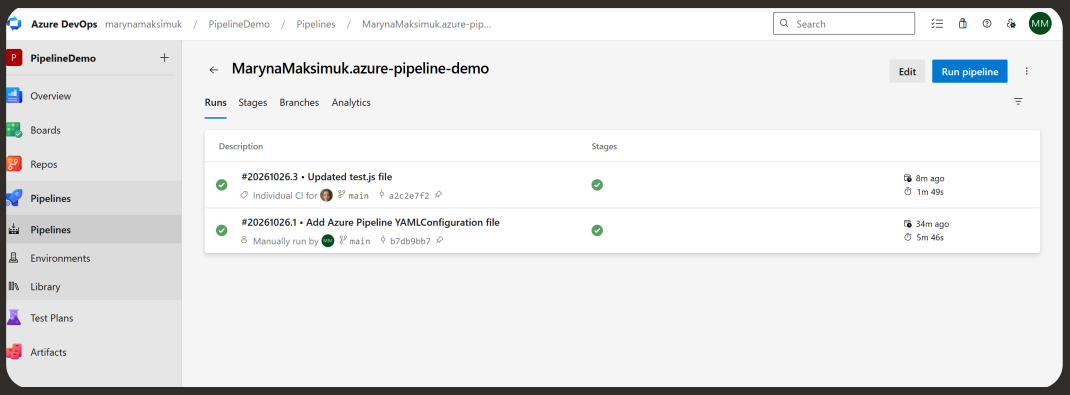
Verifying Continuous Integration End-to-End

Verification strategy

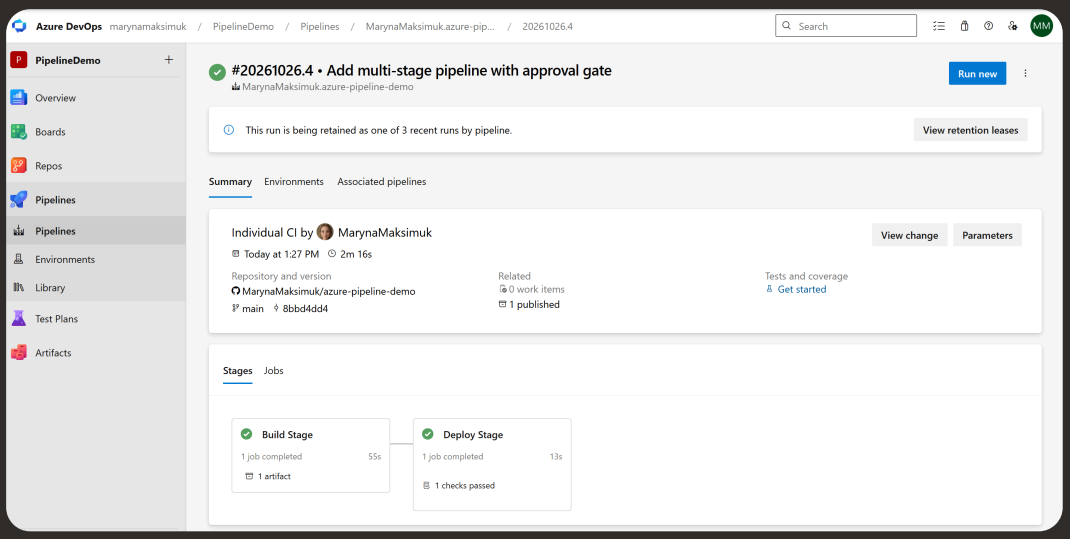
In this step, I'll verify that all stages of my Azure DevOps pipeline complete successfully, including building, testing, and publishing the artifact. I'll inspect the published artifact to confirm the expected files are included, then make a small code change and push it to GitHub to verify that the pipeline triggers automatically on the main branch.

Published artifact contents

The published nodejs-app artifact includes the Node.js application files, such as server.js, package.json, package-lock.json, and the automated test file test.js. These files package the application and its tests for download or use in later pipeline stages.



Bonus: Multi-Stage Pipeline with Manual Approval Gate



Value of approval gates in production pipelines

A manual approval gate requires a person to review and approve changes before the pipeline continues to the next stage, such as production. It helps prevent mistakes, improves security, and ensures that only authorized and tested changes are deployed.

Reflections and Key Takeaways

Tools and concepts mastered

I learned how to use Microsoft Azure and Azure DevOps, including Azure Pipelines, to build and manage CI/CD pipelines. I also learned how to set up different pipeline stages, automate testing and deployments, and use manual approval gates to control the release process.

Time and challenges

It took me around 2 hours to complete the project, including setting up the Azure DevOps pipeline, configuring different stages, and learning how manual approval gates work.

Learning goals and next steps

I completed this project today to learn how to use Azure DevOps and Azure Pipelines to create CI/CD pipelines, set up different stages, and implement manual approval gates to control deployments.